

Condicional en ensamblador x86

```
#include <stdio.h>

int main() {
    int x = 5;
    if (x > 0) {
        printf("Positive\n");
    } else {
        printf("Non-positive\n");
    }
    return 0;
}
```

El código en C declara una variable, la compara y según el resultado imprime un mensaje en pantalla

Las condicionales en x86 se implementan con comparaciones y saltos entre segmentos de código

```
.section .data
msg_pos:  .asciz "Positive\n"
msg_neg:  .asciz "Non-positive\n"

.section .text
.globl main
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $5, -4(%rbp)      # int x = 5;

    cmpl    $0, -4(%rbp)      # Compare x with 0
    jle     .else_branch      # Jump if x <= 0 (Less or Equal)

    # if branch
    leaq    msg_pos(%rip), %rdi
    call    printf
    jmp     .end_if

.else_branch:
    leaq    msg_neg(%rip), %rdi
    call    printf

.end_if:
    xorl    %eax, %eax        # return 0
    popq    %rbp
    ret
```

Ciclo en ensamblador x86

El código en C declara un contador e incrementa su valor en uno usando un ciclo que se ejecuta hasta que el contador llega a 10

```
int main() {  
    int i = 0;  
    while (i < 10) {  
        i++;           // or some work  
    }  
    return i;  
}
```

```
.globl main  
main:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    movl     $0, -4(%rbp)    # i = 0  
  
.while_condition:  
    cmpl     $10, -4(%rbp)   # compare i < 10  
    jge      .while_end      # if i >= 10, exit loop  
  
    addl     $1, -4(%rbp)    # i++  
    jmp      .while_condition # jump back to check condition again  
  
.while_end:  
    movl     -4(%rbp), %eax   # return i (which is 10)  
    popq     %rbp  
    ret
```

Para implementar un ciclo en ensamblador x86 se usa el mismo mecanismo que en una condicional, pero además se agrega un salto que solo se puede evitar si se deja de cumplir la condición del ciclo

Llamado a función en ensamblador x86

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = add(5, 7);  
    return result;  
}
```

El código en C declara una función retorna la suma de sus parámetros, y luego llama a está desde la función "main"

```
.globl add  
add:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    movl     %edi, -4(%rbp)    # first param (a)  -> edi  
    movl     %esi, -8(%rbp)    # second param (b) -> esi  
    movl     -4(%rbp), %eax  
    addl     -8(%rbp), %eax    # a + b  
    popq     %rbp  
    ret  
  
.globl main  
main:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    movl     $5, %edi          # first argument  
    movl     $7, %esi          # second argument  
    call     add               # Call instruction: pushes return address and jumps  
    movl     %eax, -4(%rbp)    # result = return value (in %eax)  
    popq     %rbp  
    ret
```

Para implementar el llamado de una función en ensamblador x86 se hace uso del stack y los registros de dirección, esto a través del uso de la instrucción "call"